

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Oberst Eduard, FAF-243

Report

Laboratory work nr. 5

The File System with Your Own Hands

Course: Operating Systems (Track A: Linux)

Checked by:
Patricia Reitman, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Purpose of the work

The purpose of this laboratory work is to check, on my own machine and with my own hands, everything the lecture said about the file system: the single tree and the file types that live in it, the inode that stands behind every name, the difference between a hard link and a symbolic link, what the permission bits really mean on a file and what they mean on a directory, and how disks, partitions and even the kernel itself appear as files. Nothing is programmed today; the work is commands, careful observation and short explanations.

All work was done on Ubuntu 26.04.1 LTS, in bash, as my normal user `capziro` and never as root, because root ignores most permission checks and the experiments of Part 3 would simply lie. This Ubuntu release ships the Rust implementations of the system utilities (uutils and sudo-rs), which is visible several times in this lab: `/usr/bin/ls` is itself a symbolic link into `/usr/lib/cargo/bin/coreutils/`, `stat` prints `size` with a small s, and the setuid hunt of the stretch part finds two copies of `sudo`.

Preparation

```
~$ whoami && mkdir -p ~/os-lab5 && cd ~/os-lab5
capziro
~/os-lab5$ script -q -a ~/os-lab5/lab5-transcript.log
```

`whoami` prints `capziro` and not `root`, so the permission experiments are honest. All the output shown below was recorded with `script` into `lab5-transcript.log`; in the figures the long prompt `capziro@capziro-3:~/os-lab5$` is shortened to `~/os-lab5$`.

Part 1. The tree and the file types

Interesting Command Output:

```
~/os-lab5$ ls /
bin   cdrom  etc   lib    lost+found  mnt  proc  run   snap  swap.img  tmp  var
boot  dev    home  lib64  media       opt  root  sbin  srv   sys       usr
```

Figure 1.1 – The root of the single tree

```
~/os-lab5$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx   1 root root     7 Apr 20 11:46 /bin -> usr/bin
crw-rw-rw-   1 root root  1, 3 Oct  9 00:30 /dev/null
drwxr-xr-x 149 root root 12288 Oct  7 18:24 /etc
-rw-r--r--   1 root root  2861 Sep  5 02:29 /etc/passwd
drwxrwxrwt  25 root root   580 Oct 12 15:17 /tmp
```

Figure 1.2 – Five paths and the first letter of ls -l

```
~/os-lab5$ ls -l /dev | head
total 0
crw-------  1 root    root     10,   262 Oct  9 00:30 acpi_thermal_rel
crw-r--r--  1 root    root     10,   235 Oct  9 00:30 autofs
drwxr-xr-x  2 root    root           720 Oct  8 22:07 block
crw-------  1 root    root     10,   234 Oct  9 00:30 btrfs-control
drwxr-xr-x  3 root    root            60 Oct  9 00:30 bus
drwxr-xr-x  2 root    root          5260 Oct 12 15:12 char
crw-------  1 root    tty       5,     1 Oct  9 00:30 console
lrwxrwxrwx  1 root    root            11 Oct  9 00:30 core -> /proc/kcore
drwxr-xr-x 18 root    root           360 Oct 12 15:12 cpu
~/os-lab5$ ls -l /dev/null /dev/nvme0n1p5
crw-rw-rw- 1 root root   1, 3 Oct  9 00:30 /dev/null
brw-rw---- 1 root disk 259, 5 Oct  8 22:07 /dev/nvme0n1p5
```

Figure 1.3 – A character device and a block device

```
~/os-lab5$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: symbolic link to ../lib/cargo/bin/coreutils/ls
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
```

Figure 1.4 – file looks at the content, not at the name

```
~/os-lab5$ du -sh /var/log /usr
du: cannot read directory '/var/log/speech-dispatcher': Permission denied
du: cannot read directory '/var/log/gdm3': Permission denied
du: cannot read directory '/var/log/chrony': Permission denied
du: cannot read directory '/var/log/private': Permission denied
du: cannot read directory '/var/log/sssd': Permission denied
207M    /var/log
9.9G    /usr
```

Figure 1.5 – The size of /var/log and /usr

Observations:

1. **For each path, its type from the first letter of ls -l.**

   The five types are collected in Table 1.1. The first character of the mode string is the type, and the nine characters after it are the permission bits of Part 3.

Table 1.1 – The type of each path

Path	First letter	Type and remark
`/etc`	`d`	Directory – 149 links, so it contains 147 subdirectories.
`/etc/passwd`	`-`	Regular file, 2861 bytes of ASCII text.
`/bin`	`l`	Symbolic link, 7 bytes long, pointing to `usr/bin`.
`/dev/null`	`c`	Character device; instead of a size it shows major 1, minor 3.
`/tmp`	`d`	Directory, but with mode `drwxrwxrwt`: the sticky bit is set.

2. **What is /bin on this system: a directory or a link, and to where?**

   `/bin` is a symbolic link to `usr/bin`, which is why its mode starts with `l` and its size is 7 – exactly the length of the string `usr/bin`. The target is written as a relative path, so it is resolved inside `/`, giving `/usr/bin`. The same is true of `/lib` and `/sbin`. This is the "usrmerge" layout: all programs and libraries really live under `/usr`, and the old top level names are kept only as links so that old paths such as `#!/bin/sh` keep working.

3. **One character device and one block device in /dev.**

   `/dev/null` is a character device (`c`) and `/dev/nvme0n1p5`, the root partition of my NVMe disk, is a block device (`b`). Where a regular file shows a size, a device file shows two numbers: the major number selects the driver in the kernel (1 for the memory devices, 259 for NVMe) and the minor number selects the concrete device (3 for `null`, 5 for the fifth partition). A character device is an unbuffered stream of bytes, a block device is addressed in fixed size blocks and can hold a file system. The file itself contains no data at all: it is only a name that carries a type and a pair of numbers.

4. **Which is bigger, /var/log or /usr, and by roughly how much?**

   `/usr` is 9.9 GB and `/var/log` is 207 MB, so `/usr` is about 49 times bigger. That is the expected ratio: `/usr` holds every installed program and library, while `/var/log` holds only text that the journal rotates and compresses. The `Permission denied` lines are themselves a result: five log directories belong to root or to a service user and my own user may not even read their names, so the 207 MB is a lower bound. Running the same command with `sudo` would give a slightly larger number.

Part 2. Inodes and links

Interesting Command Output:

```
~/os-lab5$ echo hello > notes.txt
~/os-lab5$ stat notes.txt
  File: notes.txt
  size: 6         Blocks: 8          IO Block: 4096   regular file
Device: 259,5   Inode: 18749703    Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/ capziro)   Gid: ( 1000/ capziro)
Access: 2026-10-12 15:20:46.497187070 +0300
Modify: 2026-10-12 15:20:45.827182372 +0300
Change: 2026-10-12 15:20:45.827182372 +0300
 Birth: 2026-10-12 15:20:45.827182372 +0300
~/os-lab5$ ls -li notes.txt
18749703 -rw-rw-r-- 1 capziro capziro 6 Oct 12 15:20 notes.txt
```

Figure 2.1 – An inode, and a name that is not part of it

```
~/os-lab5$ ln notes.txt hard.txt
~/os-lab5$ ls -li notes.txt hard.txt
18749703 -rw-rw-r-- 2 capziro capziro 6 Oct 12 15:20 hard.txt
18749703 -rw-rw-r-- 2 capziro capziro 6 Oct 12 15:20 notes.txt
~/os-lab5$ rm notes.txt
~/os-lab5$ cat hard.txt ; ls -li hard.txt
hello
18749703 -rw-rw-r-- 1 capziro capziro 6 Oct 12 15:20 hard.txt
```

Figure 2.2 – A hard link, and the removal of the “original”

```
~/os-lab5$ ln -s hard.txt soft.txt
~/os-lab5$ ls -li hard.txt soft.txt
18749703 -rw-rw-r-- 1 capziro capziro 6 Oct 12 15:20 hard.txt
18750216 lrwxrwxrwx 1 capziro capziro 8 Oct 12 15:21 soft.txt -> hard.txt
~/os-lab5$ cat soft.txt
hello
~/os-lab5$ rm hard.txt
~/os-lab5$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 capziro capziro 8 Oct 12 15:21 soft.txt -> hard.txt
```

Figure 2.3 – A symbolic link, and the removal of its target

```
~/os-lab5$ mkdir d ; ls -ld d
drwxrwxr-x 2 capziro capziro 4096 Oct 12 15:22 d
~/os-lab5$ mkdir d/x d/y ; ls -ld d
drwxrwxr-x 4 capziro capziro 4096 Oct 12 15:22 d
```

Figure 2.4 – Directories have links too

```
~/os-lab5$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-r-- 1 capziro capziro 2 Oct 12 15:22 one.txt
4.0K    one.txt
~/os-lab5$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-r-- 1 capziro capziro 1.0G Oct 12 15:22 big.img
0       big.img
```

Figure 2.5 – Size is not space

Observations:

1. **Which number in stat is the inode, and which tells how many names the file has?**

   `Inode: 18749703` is the inode number – the identity of the file inside the file system `259,5`. `Links: 1` is the number of names (directory entries) that point to this inode; after `ln` it became 2. `ls -li` shows the same two numbers: the inode in the first column and the link count right after the mode string. The name `notes.txt` appears in the first line of `stat` only because I typed it; it is stored in the directory, not in the inode. The size 6 is also a check on this: `echo hello` writes five letters and one newline.

2. **After rm notes.txt, why could I still read hard.txt? What does rm really remove?**

   Because both names pointed to the same inode 18749703, and the data belongs to the inode. A directory is only a table of name → inode number pairs, so `ln` added a second entry for the same inode and the link count went to 2. `rm` does not delete a file: it calls `unlink()`, which removes one entry from the directory and decreases the link count. After `rm notes.txt` the count went back to 1 and `hard.txt` was still a perfectly normal name for the same data. The inode and its blocks are freed only when the link count reaches 0 and no process still holds the file open – which is why a file that is deleted while a program is reading it keeps working until that program closes it.

3. **Why did cat soft.txt fail at the end, although soft.txt still exists? How big is it and why exactly that size?**

   A symbolic link is a file of its own, with its own inode (18750216, different from 18749703), and its content is nothing but the path of the target. `soft.txt` is 8 bytes because the text `hard.txt` is 8 characters long. When I removed `hard.txt` the last name of the real inode disappeared and the data was freed, so resolving the stored path now fails: `cat` reports `No such file or directory`, which is

about the target, not about the link. `ls -l` still lists `soft.txt` and even shows where it points, because `ls` only reads the link itself. This is the practical difference between the two kinds of link: a hard link is a second name of equal rank and can never dangle, while a symbolic link is a one way pointer that may break, but in exchange it can cross file systems and may point at a directory.

4. **A new directory has 2 links, and 4 after two subdirectories. Where does each link come from?**

   Every directory is referred to by its own `.` entry and by its name in the parent, so a fresh `d` already has two names: `d` in `~/os-lab5`, and `d/.` inside itself. Each subdirectory adds one more, because `d/x/..` and `d/y/..` both point back to `d`. With two subdirectories the count is $2 + 2 = 4$. In general the link count of a directory is 2 plus the number of its subdirectories, which is why `/etc` with 149 links contains 147 of them.

5. **Why does a 2-byte file use 4.0K, and a 1 GB file use 0?**

   `ls -l` prints the logical length of the file, `du` prints the space actually allocated to it. `one.txt` is 2 bytes (the letter and the newline), but ext4 allocates whole blocks of 4096 bytes, so the smallest non-empty file occupies 4.0K; the `Blocks: 8` line of `stat` says the same in 512-byte units. `big.img` is the opposite case: `truncate` only sets the length in the inode without writing anything, which creates a sparse file – a file with a hole. No data block is allocated, `du` reports 0, and reading the file returns zeros. Blocks are allocated only where something is really written, which is exactly how disk images and virtual machine disks stay small.

Part 3. Permissions

Interesting Command Output:

```
~/os-lab5$ echo secret > s.txt
~/os-lab5$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 capziro capziro 7 Oct 12 15:23 s.txt
~/os-lab5$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
~/os-lab5$ chmod u+r s.txt ; cat s.txt
secret
```

Figure 3.1 – chmod on a file I own

```
~/os-lab5$ umask ; touch new.txt ; mkdir newdir
0002
~/os-lab5$ ls -ld new.txt newdir
-rw-rw-r-- 1 capziro capziro    0 Oct 12 15:24 new.txt
drwxrwxr-x 2 capziro capziro 4096 Oct 12 15:24 newdir
```

Figure 3.2 – umask and the permissions of new files

```
~/os-lab5$ mkdir box ; echo hi > box/f
~/os-lab5$ chmod u=r,go= box  ; ls box ; cat box/f
f
cat: box/f: Permission denied
~/os-lab5$ chmod u=x,go= box  ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
~/os-lab5$ chmod 755 box
```

Figure 3.3 – r and x on a directory

```
~/os-lab5$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 93640 Feb  2  2026 /usr/bin/passwd
drwxrwxrwt 25 root root 580 Oct 12 15:17 /tmp
~/os-lab5$ cat /etc/shadow
cat: /etc/shadow: Permission denied
~/os-lab5$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1347 Sep  5 02:32 /etc/shadow
```

Figure 3.4 – The setuid bit, the sticky bit and another user's file

Observations:

1. **Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.**

   Each octal digit is one triad of three bits, r = 4, w = 2, x = 1, in the order owner, group, other. The four modes are given in Table 3.1.

Table 3.1 – Four common modes

Octal	rwx	A real use
600	`rw-------`	A private file: an SSH private key, `~/.netrc`, a file with a password.
644	`rw-r--r--`	A file everybody may read but only I may change: a config file, a web page, source code.
755	`rwxr-xr-x`	A program everybody may run, or a directory everybody may enter and list.
700	`rwx------`	A private directory: `~/.ssh`, `~/.gnupg`.

2. **With chmod 000 I own the file but cannot read it. Why does ownership not help?**

   Because the kernel checks the mode bits, not the feelings of the owner. On `open()` it first decides which triad applies to me – I am the owner, so the owner triad

is used, and only that one; it does not fall back to the group or other bits. With mode `000` the owner triad is empty, so reading is refused. What ownership does give me is the right to change the mode: `chmod` is a separate operation whose check is "are you the owner (or root)", not "does the mode allow it". That is why `chmod u+r s.txt` succeeded on a file I was not allowed to read one line earlier. For root the whole check is skipped, which is exactly why this lab must not be done as root.

3. **What is my umask, and how does it explain the permissions of new.txt and newdir?**

   My umask is `0002`, that is, the "write for other" bit is masked away. A program that creates a file asks for mode 666 and a program that creates a directory asks for 777; the kernel then removes every bit that is set in the umask. So `new.txt` becomes 666 and not 002 = 664 = `rw-rw-r--`, and `newdir` becomes 777 and not 002 = 775 = `rwxrwxr-x`, which is exactly what `ls -ld` shows. Note that the x bits of a directory survive while an ordinary file never gets x from the umask, because 666 does not contain them in the first place. The group write bit is harmless on Ubuntu because of user private groups: my primary group `capziro` has only me in it. On a shared server a umask of `0022` or `0077` would be used instead.

4. **Explain the box experiment: what does r allow on a directory, and what does x allow?**

   A directory is a table of name → inode pairs, and the two bits give two different rights over that table. `r` allows reading the list of names. `x` ("search") allows using a name from the table to reach the inode behind it, that is, traversing the directory on the way to something inside. With `u=r` I could run `ls box` and see the name `f`, but `cat box/f` failed, because resolving the path needs to step through `box`. With `u=x` the opposite happened: `ls box` could not open the directory to enumerate it, but `cat box/f` worked, because I already knew the name and only needed to pass through. Reading a file therefore requires x on every directory of its path and r on the file itself; this is also how a home directory with mode 711 can hide its contents while still letting a web server reach one known file inside it. Deleting a name, finally, needs w and x on the directory and says nothing about the file.

5. **What do the s in /usr/bin/passwd and the t in /tmp mean, and why does passwd need the s?**

   The `s` in `-rwsr-xr-x` is the setuid bit: the x of the owner triad is replaced by s, and whoever runs the program gets a process whose effective user is the owner of the file – here root – instead of the caller. `passwd` needs it because changing my password means writing `/etc/shadow`, and the last two lines of Figure 3.4 show why I cannot do that myself: the file is `-rw-r----- root shadow`, so my user may not even read it. The setuid bit is the controlled way to lend root's power for one well defined job, which is also why such programs are a favourite target and are kept few and small. The `t` in `drwxrwxrwt` on `/tmp` is the sticky bit: the directory is writable by everyone, but an entry may be removed or renamed only by the owner of the entry, by the owner of the directory, or by root. Without it any user could delete any other user's temporary files, since deleting depends on the permissions of the directory and not of the file.

Part 4. Disks and the kernel as files

Interesting Command Output:

```
~/os-lab5$ lsblk -e7
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
nvme0n1     259:0    0 953.9G  0 disk
|-nvme0n1p1 259:1    0   100M  0 part /boot/efi
|-nvme0n1p2 259:2    0    16M  0 part
|-nvme0n1p3 259:3    0 487.4G  0 part
|-nvme0n1p4 259:4    0   887M  0 part
`-nvme0n1p5 259:5    0 464.7G  0 part /
~/os-lab5$ lsblk -o NAME,SIZE,FSTYPE,MOUNTPOINTS -e7
NAME          SIZE FSTYPE MOUNTPOINTS
nvme0n1     953.9G
|-nvme0n1p1   100M vfat   /boot/efi
|-nvme0n1p2    16M
|-nvme0n1p3 487.4G ntfs
|-nvme0n1p4   887M ntfs
`-nvme0n1p5 464.7G ext4   /
```

Figure 4.1 – The disk, its partitions and their file systems

```
~/os-lab5$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p5  457G   35G  399G   8% /
~/os-lab5$ mount | grep " / "
/dev/nvme0n1p5 on / type ext4 (rw,relatime)
```

Figure 4.2 – How full the root file system is, and how it is mounted

```
~/os-lab5$ grep -c processor /proc/cpuinfo
16
~/os-lab5$ head -3 /proc/meminfo
MemTotal:       15545468 kB
MemFree:         1255700 kB
MemAvailable:    7305648 kB
```

Figure 4.3 – CPUs and memory, read out of /proc

```
~/os-lab5$ head -5 /proc/self/status
Name:   head
Umask:  0002
State:  R (running)
Tgid:   46267
Ngid:   0
~/os-lab5$ ls -l /proc/self/fd
total 0
lrwx------ 1 capziro capziro 64 Oct 12 15:27 0 -> /dev/pts/3
lrwx------ 1 capziro capziro 64 Oct 12 15:27 1 -> /dev/pts/3
lrwx------ 1 capziro capziro 64 Oct 12 15:27 2 -> /dev/pts/3
lr-x------ 1 capziro capziro 64 Oct 12 15:27 3 -> /proc/46271/fd
```

Figure 4.4 – /proc/self is the process that is reading

Observations:

1. **Draw the disk: name, partitions, which one is mounted at / and with what file system.**

   There is one physical disk, the NVMe SSD `nvme0n1` of 953.9 GB, with five partitions; it is a dual boot machine, so only two of them belong to Linux. The layout is given in Table 4.1. Plain `lsblk` additionally prints 27 `loop` devices mounted under `/snap`: these are not hardware at all but read-only squashfs images of the installed snap packages, attached through the loop driver (major 7), which is why Figure 4.1 uses `lsblk -e7` to exclude them.

Table 4.1 – The partitions of nvme0n1

Partition	Size	File system	Mounted at	What it is
`nvme0n1p1`	100M	`vfat`	`/boot/efi`	The EFI system partition with the boot loaders.
`nvme0n1p2`	16M	–	–	Microsoft reserved partition, not mounted.
`nvme0n1p3`	487.4G	`ntfs`	–	The Windows installation, not mounted.
`nvme0n1p4`	887M	`ntfs`	–	Windows recovery, not mounted.
`nvme0n1p5`	464.7G	`ext4`	`/`	My Ubuntu root file system.

2. **How full is the root file system?**

   `/dev/nvme0n1p5` offers 457 G, of which 35 G are used and 399 G are free: 8 % full. The 457 G of `df` are smaller than the 464.7 G of `lsblk` because `df` counts the space usable for files after the metadata (inode tables, journal, superblock copies) and the reserve for root have been taken out, and because `lsblk` counts in powers of ten while `df -h` counts in powers of two. Used and available do not add up to the size for the same reason.

3. **How many CPU cores and how much RAM, according to /proc?**

   `grep -c processor /proc/cpuinfo` counts 16 entries, that is 16 logical CPUs as the kernel schedules them (physical cores and hyper-threads are not distinguished by this count). `MemTotal` is 15 545 468 kB, about 14.8 GiB of usable RAM out of the 16 GB installed; the rest is taken by the kernel image and by memory reserved for the firmware. The interesting part is that none of this comes from a disk: `/proc` is a virtual file system, generated by the kernel on every read, so `grep` and `head` – ordinary text tools – are enough to ask the kernel about the hardware.

4. **/proc/self/status showed the name of a program. Which one, and why that one?**

   It showed `Name: head`. `/proc/self` is a magic symbolic link that the kernel resolves, for every process separately, to that process's own `/proc/<pid>` directory. The process that opened the file was not my shell but the `head` program that bash had just started, so `/proc/self` became `/proc/46267` and the status is the status of `head`. The second command shows the same effect once more: the descriptors listed are those of `ls` itself – 0, 1 and 2 on the terminal `/dev/pts/3`, plus descriptor 3, which `ls` opened to read the directory it was asked to list, and which points at `/proc/46271/fd`, the pid of that `ls`.

Stretch

(a) Magic numbers

```
~/os-lab5$ gzip -c /etc/passwd > notes.txt ; file notes.txt ; xxd notes.txt | head -1
notes.txt: gzip compressed data, was "passwd", last modified: Fri Sep  4 23:29:29 2026, from Unix,
original size modulo 2^32 2861
00000000: 1f8b 0808 5954 9b6a 0003 7061 7373 7764  ....YT.j..passwd
```

Figure 5.1 – The name says .txt, the content says gzip

The first two bytes, `1f 8b`, are the magic number of gzip; the third byte `08` is the compression method (deflate) and the fourth `08` is a flag saying that the original file name is stored in the header. That is why `file` can report not only the type but also `was "passwd"` – the name is visible in plain ASCII at offset 10 of the hex dump, and the four bytes `5954 9b6a` before it are the modification time. The extension `.txt` plays no role whatsoever: on Linux the type of a file is decided by its content, and `file` compares the first bytes against the table in `/usr/share/misc/magic`.

(b) Hunt for setuid programs

```
~/os-lab5$ find / -xdev -perm -4000 -type f 2>/dev/null
/usr/bin/ntfs-3g
/usr/bin/fusermount3
/usr/bin/gpasswd
/usr/bin/chsh
/usr/bin/umount
/usr/bin/su
/usr/bin/pkexec
/usr/bin/chfn
/usr/bin/newgidmap
/usr/bin/newuidmap
/usr/bin/passwd
/usr/bin/mount
/usr/bin/sudo.ws
/usr/sbin/pppd
/usr/lib/openssh/ssh-keysign
/usr/lib/claude-desktop/chrome-sandbox
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/cargo/bin/su
/usr/lib/cargo/bin/sudo
```

Figure 5.2 – Every setuid program on the root file system

`-perm -4000` matches files that have at least the setuid bit set, and `-xdev` keeps `find` on the root file system so that it does not walk into the 27 snap mounts. There are 19 such programs. The classics from the lecture slide are all there – `passwd`, `su`, `sudo`, `mount`, `umount`, `chsh`, `chfn`, `gpasswd` – and each of them needs one privileged operation: writing `/etc/shadow`, changing the user of a process, calling `mount()`. Four entries were not on the slide and say something about this particular laptop:

- `ntfs-3g` and `fusermount3` exist because the machine dual boots: mounting a Windows NTFS partition through FUSE as a normal user needs help from root.
- `pkexec` is the PolicyKit counterpart of `sudo` used by the graphical desktop, and `dbus-daemon-launch-helper` starts system D-Bus services; `newuidmap` and `newgidmap` set up the id ranges of unprivileged containers.

- `chrome-sandbox` belongs to an Electron application I installed (Claude Desktop). It is setuid so that the browser can build its own sandbox for renderer processes – a setuid program whose whole purpose is to give away privileges rather than to take them.
- Most interesting is that `su` and `sudo` appear twice: `/usr/bin/su` and `/usr/bin/sudo.ws` are the classic C versions, while `/usr/lib/cargo/bin/su` and `/usr/lib/cargo/bin/sudo` come from the Rust packages `rust-coreutils` and `sudo-rs`. `/usr/bin/sudo` is only a symbolic link to `/etc/alternatives/sudo`, which on Ubuntu 26.04 points at the Rust `sudo`; the old binary is kept under the name `sudo.ws` as a fallback. The same memory-safety rewrite explains the `/usr/bin/ls` link of Figure 1.4.

Conclusion

In this laboratory work I checked on my own machine what the lecture described. The most important idea is that a file and its name are two different things: the data, the size, the owner, the times and the permission bits live in the inode, while a directory is only a table that maps names to inode numbers. Everything in Part 2 follows from this one sentence. A hard link is a second entry in that table, so removing the first name changes nothing for the data and `rm` turns out to be nothing more than `unlink()` plus a counter. A symbolic link is a small file whose content is a path, which is why mine was exactly 8 bytes long and why it survived its own target. The link count of a directory stopped being mysterious once I counted `.` and the `..` of each subdirectory, and the pair 2 bytes / 4.0K and 1 GB / 0 showed that the length in the inode and the blocks on the disk are independent: ext4 allocates whole 4 KiB blocks, but a sparse file may have none at all.

Permissions turned out to be more interesting on directories than on files. On a file r, w and x mean what they say, the kernel checks exactly one triad and ownership only buys the right to call `chmod`; the `000` experiment made that obvious. On a directory the same two letters mean reading the list of names (r) and walking through it to reach an inode (x), which is why `ls box` and `cat box/f` failed in turn. The umask explained where 664 and 775 come from without anyone asking for them, and the special bits closed the picture: `passwd` is setuid root because `/etc/shadow` is unreadable for me, and `/tmp` is sticky because the right to delete a file comes from the directory and would otherwise belong to everybody.

Part 4 showed the other end of the same abstraction. Below the tree there is one NVMe disk with five partitions, of which an ext4 one is mounted at `/` and is 8 % full, and above it the kernel exposes itself as files as well: the number of cores, the amount of memory, the status and the open descriptors of a running process are all plain text under `/proc`, and `/proc/self` always means the process that is reading, which is why it reported `head` and not bash. The stretch tasks added the last two pieces – that the type of a file is decided by its first bytes and not by its extension, and that the handful of setuid programs on the system are exactly the places where the permission rules are deliberately bent, which is why they are worth knowing by name.